Final Project Report

Course Title: Compiler Lab
Course Code: CSE-3528
Session: Autumn-2025
Semester: 5th

Department of Computer Science and Engineering
International Islamic University Chittagong
Project Title: MINI CompileFix Compiler

Team Members:
1. Tahasina Tasnim Afra (C233456)
2. Nafia Nowshin (C233466)
3. Nusrath Jahan Shawon (C233449)

Supervisor: Mrs. Israt Binte Habib

Date of Submission: January 22, 2026

Contents

1 Introduction

1.1 Overview

The CompileFix Compiler is a simple console-based tool written in C++ for analyzing programs written in a small C-like language. It performs basic lexical analysis to clean the input code and break it into tokens, followed by syntax analysis to verify whether the program follows correct grammar rules. Instead of stopping at the first error, the tool attempts to continue processing the code and automatically fix common syntax mistakes.

1.2 Goals

The main goal of this project is to help beginners understand and correct their programming errors more easily. CompileFix detects frequent mistakes such as missing semicolons, unbalanced parentheses, undeclared variables, and incomplete expressions. It also keeps track of tokens, symbols, and applied fixes, and generates cleaned and corrected versions of the source code.

1.3 Scope

The scope of this project is limited to a small subset of the C programming language. It supports variable declarations, assignment statements, simple `if` statements, and basic arithmetic and relational expressions. Advanced features such as functions, arrays, pointers, preprocessing directives, and full code generation are not included in this project.

2 Background & Literature Review

2.1 Lexical analysis (scanning)

Lexical analysis is the first phase of a compiler, where the source code is read as a stream of characters and converted into meaningful units called tokens. These tokens include keywords, identifiers, numbers, operators, and symbols. During this phase, comments and extra whitespace are removed, and line and column information is recorded to help with error reporting. Many basic compiler designs described in standard textbooks use a scanner or lexer to simplify the input before further processing.

2.2 Syntax analysis (parsing)

Syntax analysis checks whether the sequence of tokens produced by lexical analysis follows the grammatical rules of the programming language. This phase organizes tokens into statements and expressions and helps detect syntax errors such as missing symbols or incorrect structure. Recursive-descent parsing is commonly used for small languages because it is easy to understand and implement, making it suitable for educational compiler projects like this one.

3 Why automatic fixing?

Most traditional compilers focus on detecting errors and reporting them to the user, but they do not attempt to correct the source code. For beginner programmers, this can make debugging

difficult, especially when multiple errors are present. Automatic fixing helps by correcting common mistakes such as missing semicolons or unbalanced parentheses and allowing the compiler to continue analysis. This approach makes the tool more user-friendly and useful for learning purposes.

4 System Design / Architecture

4.1 Pipeline

The MINI CompileFix Compiler follows a simple pipeline-based approach. The input source code is processed in multiple stages. First, the code is cleaned by removing comments and extra whitespace. Next, lexical analysis is performed to generate tokens along with line and column information. After that, syntax analysis checks the token stream against grammar rules and applies automatic fixes when possible. Finally, the system produces corrected output files and reports. The complete implementation of each stage is provided in the Appendix.

4.2 Main components

The compiler is divided into a small number of main components, each with a clear responsibility. The `EnhancedScanner` handles input reading, comment removal, and token generation. The `EnhancedParser` analyzes the tokens, checks syntax, and performs error correction. The `CodeFixer` records all applied fixes and helps generate the final corrected code. The full source code for each component and its functions is included in the Appendix.

4.3 Data structures

Basic data structures are used to support the compiler's operation. Tokens store information such as type, value, and source position. A symbol table maintains declared identifiers and their data types. Additionally, a fix list records each applied correction, including the original code and the corrected version. Details of these data structures and their implementation are shown in the Appendix.

5 Error Handling & Recovery

5.1 Targeted error types

The CompileFix Compiler focuses on handling common syntax errors that are frequently found in beginner-level programs. These include missing semicolons at the end of statements, unbalanced parentheses in expressions, and the use of undeclared identifiers. The tool also handles incomplete expressions, such as a missing operand after an operator. By targeting these frequent errors, the compiler is able to automatically correct many simple programs.

5.2 Recovery strategy

Instead of stopping execution when an error is encountered, the CompileFix Compiler uses a simple recovery strategy that attempts to fix the error and continue processing the remaining code. When a syntax problem is detected, the compiler applies the smallest possible correction,

such as inserting a missing token or adding a default value. Each fix is recorded in a log so the user can review what was changed and why. This approach allows multiple errors to be handled in a single run.

5.3 Example

To demonstrate the error handling process, consider a program containing multiple syntax errors such as missing semicolons and unbalanced parentheses. The CompileFix Compiler automatically inserts the required symbols, adds declarations for undeclared variables when possible, and completes incomplete expressions. After processing, the tool produces a corrected version of the source code along with a record of all applied fixes, making the corrections easy to understand.

6 Implementation Details

6.1 Cleaning rules

Before tokenization, the input source code is cleaned to remove unnecessary elements. Single-line comments are removed, and extra whitespace such as multiple spaces or blank lines is reduced. This cleaning step helps simplify the input and makes tokenization and parsing more reliable. The cleaned version of the code is also saved as a separate output file for reference.

6.2 Tokenization

During tokenization, the cleaned source code is scanned character by character and converted into tokens. Each token represents a keyword, identifier, number, operator, or symbol. Line and column numbers are stored with each token to support accurate error reporting. The token stream produced in this phase is used as input for the parser.

6.3 Parsing and precedence

The parser analyzes the sequence of tokens according to the defined grammar rules. A recursive-descent parsing approach is used to process statements and expressions. Operator precedence is handled by separating expressions into different levels, ensuring that multiplication and division are evaluated before addition and subtraction. This design allows expressions to be parsed correctly while keeping the implementation simple.

6.4 Outputs

After processing the input program, the CompileFix Compiler generates multiple outputs. These include a cleaned version of the source code, a fixed version with applied corrections, and printed reports such as the token table, symbol table, and fix log. These outputs help users understand both the original code and the changes made by the compiler.

7 Testing & Results

7.1 Test inputs

In addition to the basic test files, a complex error-heavy input was used to evaluate lexical cleaning, syntax fixing, and error recovery.

- `test3.c`: Contains multiple syntax errors, formatting issues, and comments.

Contents of `test3.c`:

```
int x = 5)
int m=9
int a=3
/*nxwnw
wxwqx
xexln*/
int y = 10
z = x + y)
w = x * y + z
//bjcbewjweq//
if x+2 > y {
    z = z + 1
} else {
    z = z - 1
}
/*xelknwqkl
xexed*/
```

7.2 Observed results

When `test3.c` was processed through the system, the following behaviors were observed:

- **Lexical Analysis**: Successfully removed all single-line and multi-line comments and normalized whitespace.
- **Syntax Analysis and Fixing**: Detected missing semicolons, missing parentheses in the `if` condition, and unmatched parentheses in assignment statements. The system automatically inserted missing semicolons and parentheses where possible.
- **Error Recovery**: Despite multiple syntax errors, the parser continued processing the remaining code instead of terminating early.
- **Semantic Analysis**: A symbol table was generated correctly for declared variables (`x, m, a, y`). However, detection of undeclared variables (e.g., `z`) and certain type-related issues was inconsistent, highlighting current system limitations.

7.3 Result Summary

Overall, `test3.c` demonstrates that the compiler implementation can handle heavily malformed input, recover from multiple errors, and continue analysis. While lexical and syntax

phases perform reliably, semantic analysis remains limited in complex or cascading error scenarios.

**Limitations:* Some complex scenarios, such as multiple variable declarations in a single statement (e.g., `int a, b;`), may not be fully recognized. Type checking for expressions with mixed types is basic and may produce warnings rather than strict enforcement.

7.4 Sample Output Snapshots

- `cleaned_code.c`: shows code after comment removal and whitespace normalization.
- `tokens.txt`: lists tokens line by line with type and line number.
- `syntax_fixed.c`: shows recovered code after auto-fixing semicolons, parentheses, and braces.
- `semantic_report.txt`: shows the symbol table and detected semantic errors/warnings.

8 Limitations and Future Improvements

8.1 Limitations of the Mini Compiler

Although the proposed mini compiler successfully performs lexical analysis, syntax analysis, and basic semantic checking, it has several limitations due to its simplified design and restricted scope. The major limitations are listed below:

- **Limited declaration handling:** The semantic analyzer does not fully support multiple variable declarations in a single statement such as:

  ```
  int a, b;
  ```

  In some cases, the second (or subsequent) variable may be reported as undeclared because the analysis is performed sequentially without a complete declarator-list grammar.
- **No full grammar parsing:** The compiler does not employ a formal LL(1) or LR(1) parser. Instead, it relies on hand-crafted, pattern-based recursive descent with limited lookahead, which can fail for complex or deeply nested language constructs and certain ambiguous cases.
- **Limited expression evaluation:** Type checking uses a simple dominant-type (or promotion-based) approach rather than full expression tree evaluation. Consequently, complex expressions involving mixed operators, implicit conversions, or operator precedence issues may not always be analyzed correctly.
- **No scope resolution:** Block-level or nested scopes are not supported. All variables are effectively treated as having global (file-level) scope, which leads to incorrect semantic behavior when variables are redeclared or shadowed inside $\{\}$ blocks.
- **No function semantic analysis:** The compiler does not perform any validation of function declarations, parameter matching, return type consistency, or function call compatibility (including argument count and type checking).

- **Limited error recovery:** Although basic phrase-level and panic-mode error recovery techniques are implemented for syntax errors, recovery is not always graceful. Heavily malformed programs can produce cascading errors or misleading diagnostic messages.
- **No support for arrays, pointers, or structures:** Advanced C-like language features such as arrays (including indexing), pointers (including dereferencing and pointer arithmetic), and user-defined structures/unions are completely unsupported in both syntax and semantic phases.
- **Token-based semantic analysis:** Semantic checks are performed directly on the stream of tokens rather than on a structured intermediate representation such as an Abstract Syntax Tree (AST). This severely limits the ability to accurately represent and analyze complex program structures and control flow.
- **No code generation or optimization:** The current implementation stops after semantic analysis and does not produce target machine code, intermediate code (e.g., three-address code), or any form of executable output.
- **Lack of standard library / built-in functions:** No built-in functions (e.g., `printf`, `scanf`) or preprocessor directives are recognized or handled, restricting the compiler to purely computational toy programs.
- **No compile-time constant evaluation:** Constant expressions are not folded or evaluated at compile time, which limits both optimization potential and some forms of semantic checking.

8.2 Future Improvements

The following enhancements can be made to overcome the current limitations and evolve the mini compiler into a more robust and feature-complete system:

- Implementing a formal parser using LL(1) or LR(1) parsing techniques (e.g., with tools such as ANTLR, Bison, or hand-written recursive descent with full FIRST/FOLLOW sets).
- Constructing and traversing an Abstract Syntax Tree (AST) to enable more accurate and powerful semantic analysis, type checking, and future code generation phases.
- Adding proper lexical scoping using a stack of symbol tables (push on entering a block, pop on exiting) to correctly handle variable shadowing and block-level declarations.
- Supporting multiple variable declarations in a single statement by improving the grammar rule for declarator lists and updating the symbol table insertion logic accordingly.
- Extending semantic analysis to support function declarations, function definitions, parameter type checking, return type validation, and function call resolution.
- Adding support for arrays (declaration, indexing, bounds checking where possible) and pointers (basic pointer declaration, dereferencing, and address-of operations).
- Implementing a simple code generation backend (e.g., producing LLVM IR, x86 assembly, or C code as output) to make the compiler capable of creating executable programs.

- Improving error reporting and recovery mechanisms with better diagnostic messages, error categorization (warning vs. error), and more sophisticated recovery strategies (e.g., error productions, phrase-level repair).
- Introducing basic support for preprocessor directives (`#define`, `#include`) and simple macro expansion.
- Adding constant folding and basic compile-time expression evaluation to improve both semantic checking and generated code quality.
- Creating a more modular architecture (separate lexer, parser, semantic analyzer, and code generator modules) to make future extensions easier.

9 Conclusion

This project demonstrates a practical compiler front-end style workflow that includes source code cleaning, lexical analysis, syntax checking, and basic error recovery. The generated fix reports and corrected output files make the tool useful both for understanding core compiler concepts and for improving poorly formatted or error-prone student code. Overall, the project highlights how simple compiler techniques can be applied to automatically detect and correct common programming mistakes.

10 Appendix

SOURCE CODE: (header + isNumberToken function)

```cpp
#include <bits/stdc++.h>
#include <fstream>
#include <string>
#include <cctype>
#include <vector>
#include <algorithm>
#include <map>
using namespace std;

bool isNumberToken(const string& token)
{
    bool dotSeen = false;

    for (char c : token)
    {
        if (c == '.') // for floating numbers
        {
            if (dotSeen) return false;
            dotSeen = true;
        }
        else if (!isdigit(c))
            return false;
    }

    return !token.empty();
}
```

(lex)

```cpp
void lexicalPhase()
{
    ifstream in("test3.c");
    ofstream cleanOut("cleaned_code.c");
    ofstream tokenOut("tokens.txt");
```

```

    if (!in || !cleanOut || !tokenOut)
    {
        cout << "File error in lexical phase.\n";
        return;
    }

    string code((istreambuf_iterator<char>(in)), istreambuf_iterator<char>());
    string cleanCode = "";

    // ---------- COMMENT REMOVAL (SAFE VERSION) ----------
    bool inString = false;

    for (size_t i = 0; i < code.length(); i++)
    {
        char current = code[i];
        char next = (i + 1 < code.length()) ? code[i + 1] : '\0';

        // toggle string mode (ignore escaped quotes)
        if (current == '"' && (i == 0 || code[i - 1] != '\\'))
        {
            inString = !inString;
            cleanCode += current;
            continue;
        }

        // remove single-line comments (only if not in string)
        if (!inString && current == '/' && next == '/')
        {
            while (i < code.length() && code[i] != '\n')
                i++;
            cleanCode += '\n'; // keep line structure
            continue;
        }

        // remove multi-line comments (only if not in string)
        if (!inString && current == '/' && next == '*')
        {
            i += 2;
            while (i + 1 < code.length() &&
                !(code[i] == '*' && code[i + 1] == '/'))
            {
                i++;
            }
            i++; // skip '/'
            continue;
        }

        // normal character
        cleanCode += current;
    }

    cleanOut << cleanCode;

    // ---------- TOKENIZATION ----------
    int lineNo = 1;
    string token = "";
    string operators = "+-*/=<>!";     // single-char operators
    string separators = "();{},";      // separators like ;, (, {, )
    unordered_set<string> keywords = {
        "int","float","char","if","else","for","while","return"
    };

    for (size_t i = 0; i < cleanCode.length(); i++)
    {
        char ch = cleanCode[i];

        if (ch == '\n') lineNo++;

        // ---- Handle whitespace ----
        if (isspace(ch))
        {
            if (!token.empty())
```

```
            {
                // check if keyword
                if (find(keywords.begin(), keywords.end(), token) != keywords.end())
                    tokenOut << "KEYWORD: " << token << " : " << lineNo << endl;

                else if (isNumberToken(token))
                    tokenOut << "NUMBER: " << token << " : " << lineNo << endl;

                else
                    tokenOut << "IDENTIFIER: " << token << " : " << lineNo << endl;


                token.clear();
            }
            continue;
        }

        // ---- Handle separators ----
        if (separators.find(ch) != string::npos)
        {
            if (!token.empty())
            {
                // flush current token
                if (find(keywords.begin(), keywords.end(), token) != keywords.end())
                    tokenOut << "KEYWORD: " << token << " : " << lineNo << endl;

                else if (isNumberToken(token))
                    tokenOut << "NUMBER: " << token << " : " << lineNo << endl;

                else
                    tokenOut << "IDENTIFIER: " << token << " : " << lineNo << endl;


                token.clear();
            }
            tokenOut << "SEPARATOR: " << ch << " : " << lineNo << endl;

            continue;
        }

        // ---- Handle operators (multi-char) ----
        if (operators.find(ch) != string::npos)
        {
            if (!token.empty())
            {
                if (find(keywords.begin(), keywords.end(), token) != keywords.end())
                    tokenOut << "KEYWORD: " << token << " : " << lineNo << endl;

                else if (isNumberToken(token))
                    tokenOut << "NUMBER: " << token << " : " << lineNo << endl;

                else
                    tokenOut << "IDENTIFIER: " << token << " : " << lineNo << endl;


                token.clear();
            }

            // check for two-character operators like ==, >=, <=, !=, ++, --
            string op(1, ch);
            if (i + 1 < cleanCode.length())
            {
                char nextCh = cleanCode[i + 1];
                if ((ch == '+' && nextCh == '+') || (ch == '-' && nextCh == '-') ||
                        (ch == '=' && nextCh == '=') || (ch == '!' && nextCh == '=') ||
                        (ch == '<' && nextCh == '=') || (ch == '>' && nextCh == '='))
                {
                    op += nextCh;
                    i++; // skip next char
                }
            }

            tokenOut << "OPERATOR: " << op << " : " << lineNo << endl;
```

```

            continue;
        }

        // ---- Otherwise accumulate token ----
        token += ch;
    }

// ---- flush remaining token ----
    if (!token.empty())
    {
        if (find(keywords.begin(), keywords.end(), token) != keywords.end())
            tokenOut << "KEYWORD: " << token << " : " << lineNo << endl;

        else if (isNumberToken(token))
            tokenOut << "NUMBER: " << token << " : " << lineNo << endl;

        else
            tokenOut << "IDENTIFIER: " << token << " : " << lineNo << endl;

    }


    cout << "Lexical Analysis Completed.\n";
    cout << "output done: cleaned_code.c\n";
    cout << "output done: tokens.txt\n";
}
```

(parser)

```
void syntaxPhase()
{
    ifstream in("cleaned_code.c");
    ofstream fixed("syntax_fixed.c");
    ofstream errors("syntax_errors.txt");

    if (!in || !fixed || !errors)
    {
        cout << "Syntax phase file error.\n";
        return;
    }

    string line;
    int lineNo = 1;
    int openParen = 0, openBrace = 0;

    while (getline(in, line))
    {
        string newLine = "";
        int localParen = 0; // per-line parentheses

        // ---- FIX EXTRA ')' ----
        for (char c : line)
        {
            if (c == '(')
            {
                localParen++;
                openParen++;
                newLine += c;
            }
            else if (c == ')')
            {
                if (localParen > 0)
                {
                    localParen--;
                    openParen--;
                    newLine += c;
                }
                else
                {
                    errors << "Line " << lineNo
                           << ": Extra ')' removed.\n";
```

```
                // skip extra ')'
            }
        }
        else
        {
            newLine += c;
        }
    }

    line = newLine;

    // ---- FIX MISSING ')' IN SAME STATEMENT (PHRASE LEVEL) ----
    int countOpen = 0, countClose = 0;
    for (char c : line)
    {
        if (c == '(') countOpen++;
        if (c == ')') countClose++;
    }

    if (countOpen > countClose &&
            line.find("if") == string::npos &&
            line.find("for") == string::npos &&
            line.find("while") == string::npos)
    {

        int missing = countOpen - countClose;
        while (missing--)
        {
            line += ")";
            openParen--; // prevent panic-mode adding it later
            errors << "Line " << lineNo
                   << ": Missing ')' in statement. Auto-fixed.\n";
        }
    }


    // ---- FIX IF CONDITION (missing parentheses) ----
    if (line.find("if") != string::npos &&
            line.find("(") == string::npos &&
            line.find(")") == string::npos)
    {

        size_t ifPos = line.find("if");
        size_t bracePos = line.find("{");

        if (bracePos != string::npos)
        {
            string condition = line.substr(ifPos + 2, bracePos - (ifPos + 2));
            line = "if (" + condition + ") {";

            errors << "Line " << lineNo
                   << ": Missing parentheses in if-condition. Auto-fixed.\n";
        }
    }

    // ---- COUNT BRACES ----
    for (char c : line)
    {
        if (c == '{') openBrace++;
        if (c == '}') openBrace--;
    }

    // ---- FIX MISSING SEMICOLON ----
    if (!line.empty() &&
            line.find("if") == string::npos &&
            line.find("else") == string::npos &&
            line.back() != ';' &&
            line.back() != '{' &&
            line.back() != '}')
    {

        line += ";";
        errors << "Line " << lineNo
```

```
                    << ": Missing semicolon. Auto-fixed.\n";
        }

        fixed << line << endl;
        lineNo++;
    }

    // ---- PANIC MODE: MISSING CLOSING SYMBOLS ----
    while (openParen > 0)
    {
        fixed << ")" << endl;
        errors << "Panic Mode: Missing ')'. Inserted.\n";
        openParen--;
    }

    while (openBrace > 0)
    {
        fixed << "}" << endl;
        errors << "Panic Mode: Missing '}'. Inserted.\n";
        openBrace--;
    }

    cout << "Syntax Analysis Completed.\n";
    cout << "Output done: syntax_fixed.c\n";
    cout << "Output done: syntax_errors.txt\n";
}
```

(semantic)

```
void semanticPhase() {
    ifstream in("tokens.txt");
    ofstream report("semantic_report.txt");

    if (!in || !report) {
        cout << "Semantic phase file error.\n";
        return;
    }

    map<string, string> symbolTable; // variable -> type
    vector<string> errors;

    string currentType = "";
    bool declarationMode = false;
    set<string> declaredThisStatement; // identifiers in current statement
    string lastAssignedType = "";
    string lastTokenType = "";
    string lastTokenValue = "";

    string line;
    while (getline(in, line)) {
        if (line.empty()) continue;

        // Expected format: TYPE: value : lineNo
        size_t firstColon = line.find(":");
        size_t secondColon = line.rfind(":");

        if (firstColon == string::npos ||
            secondColon == string::npos ||
            firstColon == secondColon)
            continue;

        string tokenType = line.substr(0, firstColon);
        string tokenValue = line.substr(firstColon + 1,
                                        secondColon - firstColon - 1);
        int tokenLine = stoi(line.substr(secondColon + 1));

        // trim spaces
        tokenType.erase(remove(tokenType.begin(), tokenType.end(), ' '),
                        tokenType.end());
        tokenValue.erase(0, tokenValue.find_first_not_of(" "));
        tokenValue.erase(tokenValue.find_last_not_of(" ") + 1);

```

```
        // ---- Type keywords ----
        if (tokenType == "KEYWORD") {
            if (tokenValue == "int" || tokenValue == "float" || tokenValue == "char") {
                currentType = tokenValue;
                declarationMode = true;
                declaredThisStatement.clear();
            } else {
                declarationMode = false; // if, else, return, etc
                currentType = "";
                declaredThisStatement.clear();
            }
            lastTokenType = tokenType;
            lastTokenValue = tokenValue;
            continue;
        }

        // ---- Numbers ----
        if (tokenType == "NUMBER") {
            if (tokenValue.find('.') != string::npos)
                lastAssignedType = "float";
            else
                lastAssignedType = "int";
            lastTokenType = tokenType;
            lastTokenValue = tokenValue;
            continue;
        }

        // ---- Identifiers ----
        if (tokenType == "IDENTIFIER") {
            string varName = tokenValue;

            if (declarationMode && (lastTokenType == "KEYWORD" || (lastTokenType == "SEPARATOR" &&
                 lastTokenValue == ","))) {
                // It's a declaration
                if (declaredThisStatement.count(varName)) {
                    errors.push_back("Error at line " + to_string(tokenLine) +
                                     ": Multiple declaration of " + varName);
                } else if (symbolTable.count(varName)) {
                    errors.push_back("Error at line " + to_string(tokenLine) +
                                     ": Variable already declared previously");
                } else {
                    symbolTable[varName] = currentType;
                    declaredThisStatement.insert(varName);
                }
            } else {
                // It's a use
                declarationMode = false; // exit declaration mode if it was on
                if (!symbolTable.count(varName)) {
                    errors.push_back("Semantic Error at line " + to_string(tokenLine) +
                                     ": Undeclared variable " + varName);
                } else {
                    string varType = symbolTable[varName];
                    if (!lastAssignedType.empty() && varType == "int" && lastAssignedType == "float") {
                        errors.push_back("Warning at line " + to_string(tokenLine) +
                                     ": Assigning float to int may lose data");
                    }
                }
            }
            lastTokenType = tokenType;
            lastTokenValue = tokenValue;
            continue;
        }

        // ---- Separators ----
        if (tokenType == "SEPARATOR") {
            if (tokenValue == ";") {
                declarationMode = false;
                currentType = "";
                declaredThisStatement.clear();
                lastAssignedType = "";
            }
            lastTokenType = tokenType;
            lastTokenValue = tokenValue;
```

```
            continue;
        }

        // ---- Ignore operators ----
        if (tokenType == "OPERATOR") {
            lastTokenType = tokenType;
            lastTokenValue = tokenValue;
            continue;
        }
    }

    // ---- Output errors ----
    for (auto &e : errors) report << e << endl;

    // ---- Output symbol table ----
    report << "\nSymbol Table:\n";
    for (auto &s : symbolTable)
        report << s.first << " : " << s.second << endl;

    cout << "Semantic Analysis Completed.\n";
    cout << "Output done: semantic_report.txt\n";
}
```

(main)

```
int main()
{
    int choice;

    cout << "====================================\n";
    cout << "      MINI CompileFix COMPILER      \n";
    cout << "====================================\n";

    do
    {
        cout << "\n------------ MAIN MENU --------------\n";
        cout << "1. Lexical Analysis\n";
        cout << "  - Remove Comments\n";
        cout << "  - Tokenization\n";
        cout << "2. Syntax Analysis\n";
        cout << "  - Phrase Level Recovery\n";
        cout << "  - Panic Mode Recovery\n";
        cout << "3. Semantic Analysis\n";
        cout << "  - Symbol Table\n";
        cout << "  - Type Checking\n";
        cout << "4. Exit\n";
        cout << "-------------------------------------\n";
        cout << "Enter your choice: ";
        cin >> choice;

        switch (choice)
        {
        case 1:
            cout << "\n>>> Starting Lexical Analysis...\n";
            lexicalPhase();
            cout << ">>> Lexical Analysis Finished.\n";
            break;

        case 2:
            cout << "\n>>> Starting Syntax Analysis...\n";
            syntaxPhase();
            cout << ">>> Syntax Analysis Finished.\n";
            break;

        case 3:
            cout << "\n>>> Starting Semantic Analysis...\n";
            semanticPhase();
            cout << ">>> Semantic Analysis Finished.\n";
            break;

        case 4:
            cout << "\nExiting Mini Compiler. Goodbye!\n";
```

```
            break;

        default:
            cout << "\nInvalid choice! Please try again.\n";
        }

    }
    while (choice != 4);
    return 0;
}
```